BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ

MÔN HỌC

HỆ THỐNG NHÚNG

GIAO TIẾP SPI GIỮA MODULE RFID VÀ STM32

MÃ MÔN HỌC: EMSY435664

SÁNG THỨ 6, ĐỢT 2, HỌC KÌ 1, NĂM HỌC 2025-2026

Thực hiện:

Giang Vĩnh Huy 23119146

Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Hà

Thành phố Hồ Chí Minh, Tháng 1 năm 2026

Phụ lục

1. Giao tiếp SPI giữa module RFID và STM32F103C8T6 1

1.1. Định nghĩa 1

1.2. Đặc điểm 1

1.3. Nguyên lí hoạt động 2

1.4. Cấu trúc gói tin và dạng sóng 3

1.5. Lưu đồ trình tự 5

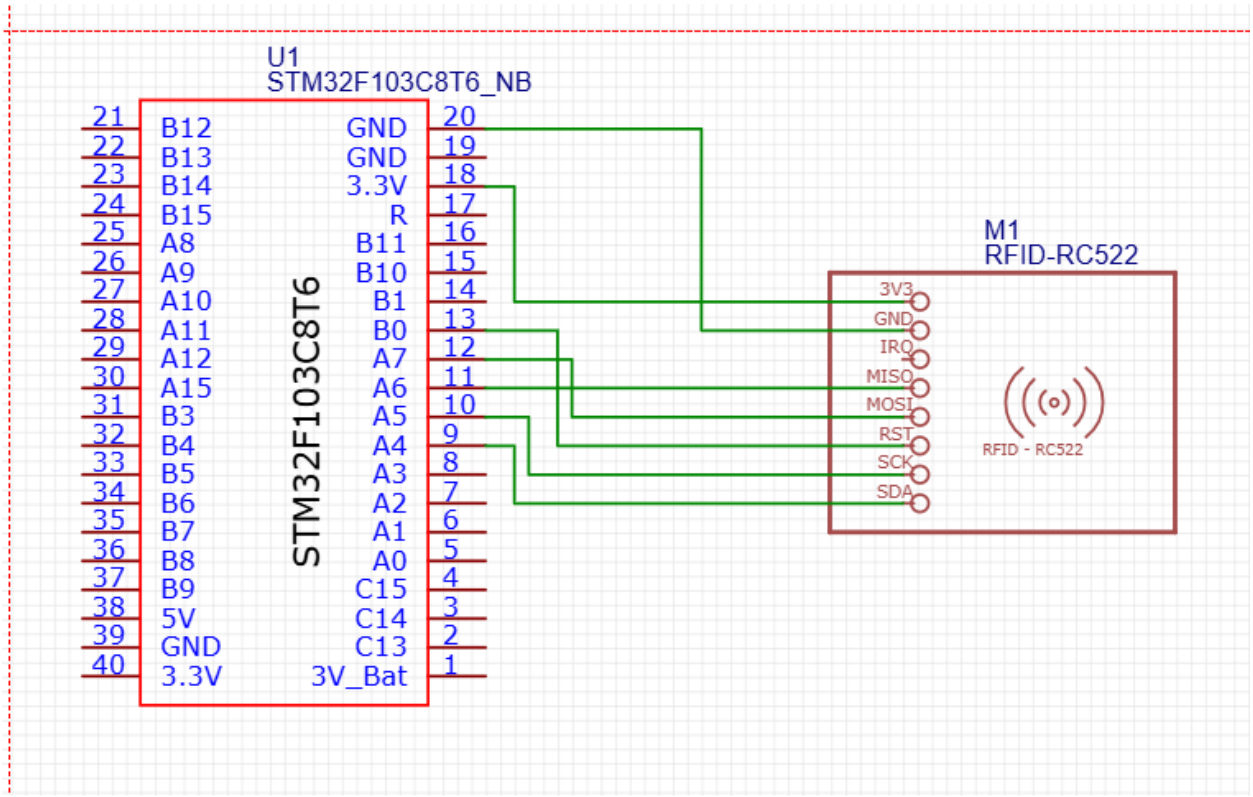
1.6. Chi tiết quy trình 6

a. Giải thích trên STM32CUBEMX 6

b. Giải thích code 7

1. Giao tiếp SPI giữa module RFID và STM32F103C8T6

Mạch schematic:



Schematic mạch giao tiếp SPI giữa RFID và STM32F103C8T6

1.1. Định nghĩa

RFID (Radio Frequency Identification) là công nghệ nhận dạng và thu thập dữ liệu tự động thông qua sóng vô tuyến. Công nghệ này cho phép xác định, theo dõi và quản lý đối tượng bằng cách sử dụng thẻ RFID mà không cần tiếp xúc trực tiếp giữa thẻ và thiết bị đọc

Hệ thống RFID được ứng dụng rộng rãi trong các lĩnh vực như quản lý ra vào, quản lý kho, truy xuất nguồn gốc sản phẩm và các hệ thống tự động hóa

1.2. Đặc điểm

RFID có một số đặc điểm sau:

- Tần số hoạt động: 13.56 MHz

- Phạm vi đọc: từ 0 – 5 cm

- Phương thức giao tiếp: không dây, sử dụng sóng vô tuyến

- Chuẩn giao tiếp phổ biến: ISO/IEC 14443 (RFID HF)

- Tốc độ truyền dữ liệu: lên đến 106 kbps

- Khả năng đọc: đọc đơn hoặc đọc đồng thời nhiều thẻ

- Loại thẻ: thẻ thụ động

- Nguồn cấp cho thẻ: không sử dụng pin, lấy năng lượng từ sóng của đầu đọc

- Dữ liệu lưu trữ: mã định danh duy nhất (UID) và vùng nhớ người dùng

- Độ bền: cao, không có tiếp điểm cơ học

- Ứng dụng tiêu biểu: thẻ sinh viên, kiểm soát ra vào, chấm công

1.3. Nguyên lí hoạt động

Hệ thống hoạt động dựa trên nguyên lý giao tiếp Master-Slave thông qua chuẩn kết nối SPI. Trong đó, vi điều khiển STM32 đóng vai trò Master điều phối toàn bộ quá trình, còn module RC522 đóng vai trò Slave thực hiện các thao tác vật lý liên quan đến sóng vô tuyến tần số 13.56MHz

Đối với giao thức SPI trong hệ thống này, dữ liệu được truyền nhận song song trên hai đường MOSI và MISO dưới sự đồng bộ của xung nhịp SCK. Quá trình giao tiếp bắt đầu khi chân Chip Select được STM32 kéo xuống mức logic thấp. Gói tin truyền đi tuân theo cấu trúc frame quy định: Byte đầu tiên luôn là byte địa chỉ thanh ghi, trong đó bit cao nhất quyết định chế độ truy cập. Nếu bit này là 0, hệ thống thực hiện thao tác Ghi; nếu bit này là 1, hệ thống thực hiện thao tác Đọc

Cơ chế hoạt động của module RC522 dựa trên việc thao tác trực tiếp với các thanh ghi nội bộ. Để kích hoạt khả năng đọc thẻ, trước hết STM32 cần cấu hình các thanh ghi quan trọng như: thanh ghi MFRC522_REG_T_MODE(cái 2ah) và MFRC522_REG_T_PRESCALER(cái 2bh) để thiết lập bộ định thời, và quan trọng nhất là thanh ghi MFRC522_REG_TX_CONTROL để bật antenna, tạo ra từ trường năng lượng cho thẻ hoạt động

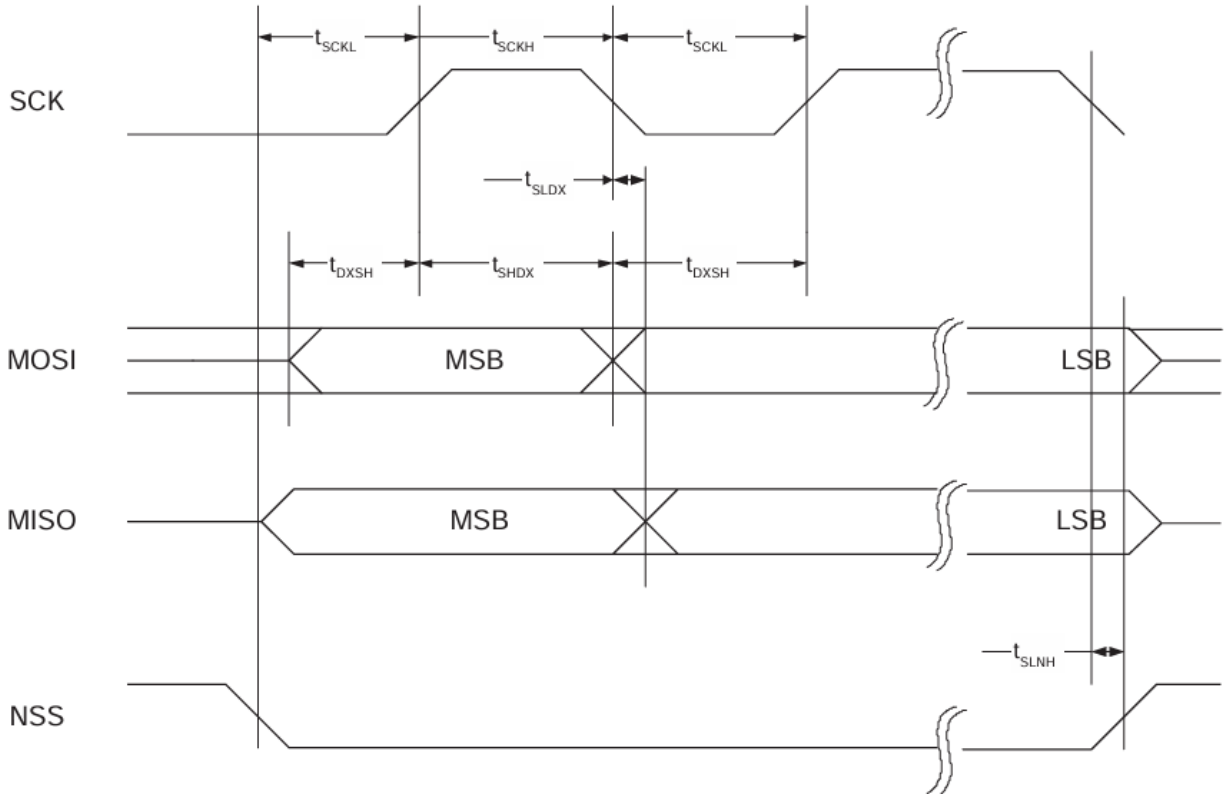
Quy trình giao tiếp gồm 3 giai đoạn chính:

- Giai đoạn tìm kiếm: STM32 gửi lệnh PICC_REQIDL (0x26) xuống RC522. Module sẽ phát sóng để đánh thức các thẻ đang ở trạng thái nghỉ (Idle) trong vùng phủ sóng

- Giai đoạn chống va chạm: Đây là bước quan trọng nhất để định danh thẻ. STM32 gửi lệnh PICC_ANTICOLL (0x93). Tại bước này, thẻ sẽ phản hồi về mã định danh duy nhất (UID). Module RC522 nhận dữ liệu này và STM32 sẽ đọc về thông qua bộ đệm FIFO. Để đảm bảo tính toàn vẹn dữ liệu, một thuật toán Checksum (XOR các byte UID) được thực hiện; nếu kết quả khớp với byte kiểm tra (BCC) thì thẻ mới được coi là hợp lệ

- Giai đoạn ngưng hoạt động: Sau khi đã đọc được UID, lệnh PICC_HALT (0x50) được gửi đi để đưa thẻ vào trạng thái ngủ, ngăn chặn việc đọc lặp lại liên tục khi thẻ vẫn nằm trên đầu đọc

1.4. Cấu trúc gói tin và dạng sóng



Cấu trúc dạng sóng

Table 8. Address byte 0 register; address MOSI

7 (MSB)	6	5	4	3	2	1	0 (LSB)
1 = read 0 = write	address						0

Cấu trúc gói tin giao tiếp SPI

Cấu trúc gói tin gồm:

- Bit 7: Bit quy định hướng, đây là bit quy định hành động

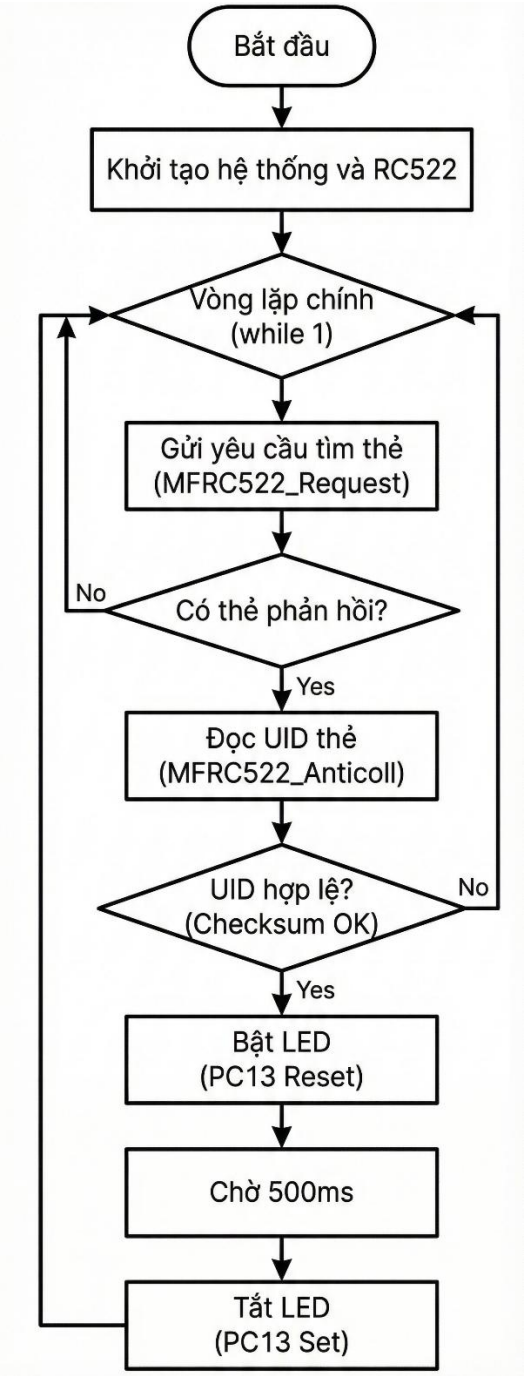
    + Giá trị = 0: Lệnh ghi và STM32 sẽ gửi thêm dữ liệu để cài đặt chip

    + Giá trị = 1: Lệnh đọc và RC522 sẽ gửi trả dữ liệu về cho STM32

- Bit 6 đến bit 1: Địa chỉ thanh ghi

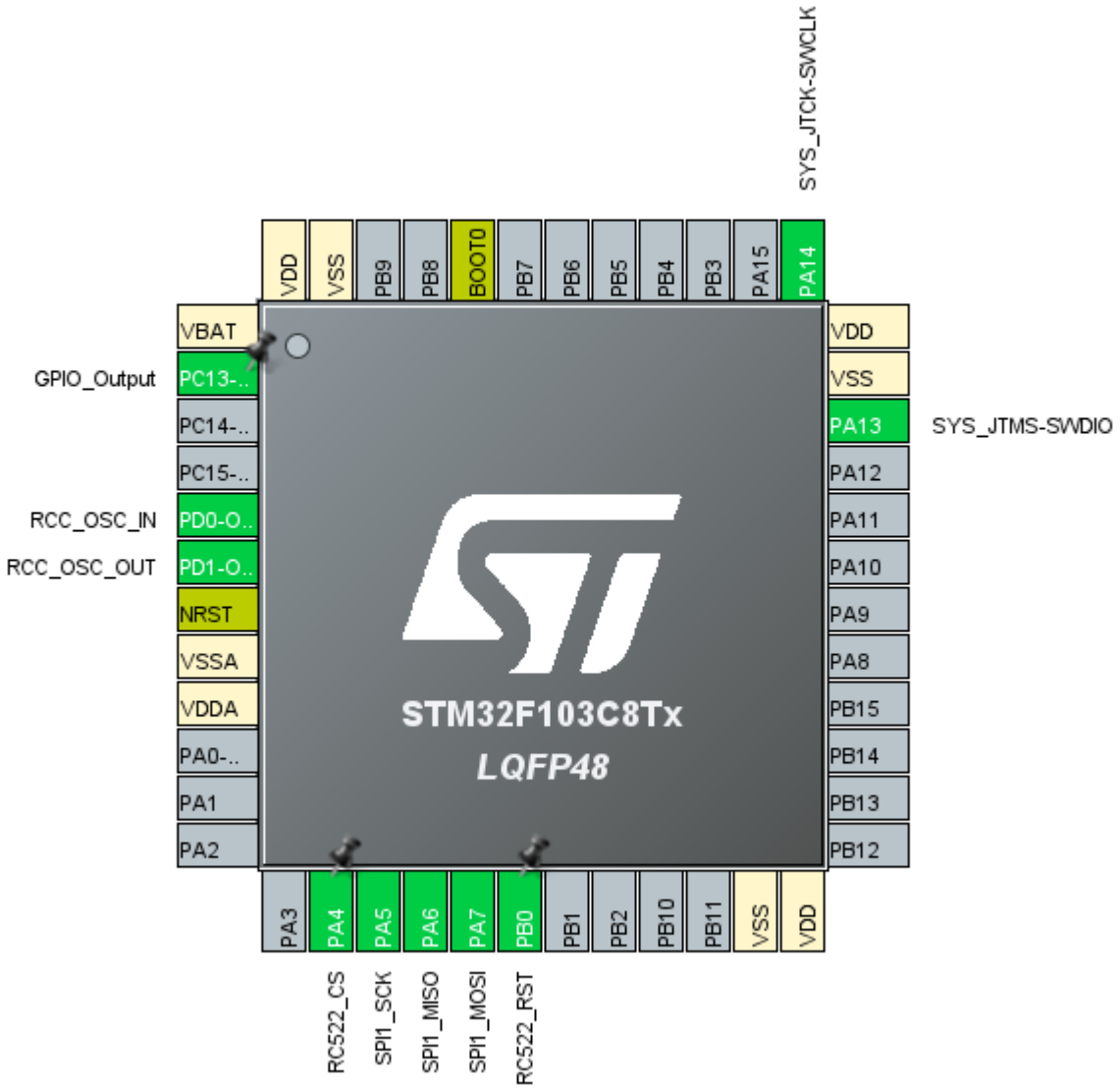
- Bit 0: Bit chốt, trong datasheet quy định bit này luôn luôn phải bằng 0. Nó đóng vai trò như một ký tự kết thúc định dạng địa chỉ

1.5. Lưu đồ trình tự



1.6. Chi tiết quy trình

a. Giải thích trên STM32CUBEMX



Giải thích các chân ở trên:

- PA5 (spi1_sck) là chân tạo xung nhịp cho giao tiếp spi. vi điều khiển sẽ phát xung để đồng bộ dữ liệu với module rfid

- PA6 (spi1_miso) là xchân master in slave out. chân này nhận dữ liệu từ module rfid gửi về vi điều khiển stm32

- PA7 (spi1_mosi) là chân master out slave in. chân này gửi dữ liệu từ stm32 xuống module rfid

- PA4 là chân chọn chip select dùng để điều khiển, để sử dụng được thì ta cần kéo chân xuống mức thấp

- PB0 là chân dùng để thực hiện chức năng reset

- PA13 (sys_jtms-swdio) là chân serial wire debug input output nó là chân truyền tải dữ liệu nạp code và debug hai chiều.

- PA14 (sys_jtck-swclk) là chân serial wire clock nó là chân xung nhịp đồng bộ cho quá trình nạp/debug.

- PD0 (RCC_OSC_IN) là chân vào của bộ dao động thạch anh cao tốc (HSE).

- PD1 (RCC_OSC_OUT) là chân ra của bộ dao động thạch anh.

- PC13 là chân cho led dùng để hiển thị khi quét thẻ và đọc được ID

b. Giải thích code

Giải thích file rc522.h

```
#ifndef __RC522_H

#define __RC522_H
```

- Những dòng này có tác dụng là kĩ thuật header guard. Nó có tác dụng đảm bảo nội dung file chỉ được biên dịch đúng 1 lần

```
#ifdef __cplusplus
 extern "C" {
#endif
```

- Những dòng này có tác dụng giúp file header có thể sử dụng được cả dự án bằng ngôn ngữ C và C++

```
#define RC522_CS_GPIO_Port  GPIOA
#define RC522_CS_Pin        GPIO_PIN_4
#define RC522_RST_GPIO_Port GPIOB
#define RC522_RST_Pin       GPIO_PIN_0
```

- Những dòng này có tác dụng định nghĩa các chân kết nối theo sơ đồ CubeMx ban đầu, 2 dòng đầu ở trên định nghĩa RC522_CS nằm ở PA4 còn 2 dòng sau định nghĩa RC522_RST nằm ở PB0

```
#define MI_OK           0
#define MI_NOTAGERR     1
#define MI_ERR          2
```

- Những dòng trên định nghĩa trạng thái trả về:

+ MI_OK (0): trạng thái thành công; mọi thứ hoạt động bình thường

+ MI_NOTAGERR (1): lỗi không tìm thấy thẻ; thường xảy ra khi không có thẻ nào nằm trong vùng quét của module

+ MI_ERR (2): lỗi chung; có thể do đường truyền dữ liệu bị nhiễu hoặc lỗi logic trong quá trình xử lý

```
#define PCD_IDLE        0x00
#define PCD_AUTHENT     0x0E
#define PCD_RECEIVE     0x08
#define PCD_TRANSMIT    0x04
#define PCD_TRANSCEIVE  0x0C
#define PCD_RESETPHASE  0x0F
#define PCD_CALCCRC     0x03
```

- Những dòng trên thì là các mã lệnh gửi cho chip RC522, em tra được trong data sheet như sau:

18.3 MFRC522 Commands Overview

Table 149: Command overview

Command	Command code	Action
Idle	0000	No action; cancels current command execution.
Mem	0001	Stores 25 byte into the internal buffer
Generate RandomID	0010	Generates a 10 byte random ID number
CalcCRC	0011	Activates the CRC co-processor or performs a selftest.
Transmit	0100	Transmits data from the FIFO buffer.
NoCmd Change	0111	No command change. This command can be used to modify different bits in the command register without touching the command. E.g. Power-down.
Receive	1000	Activates the receiver circuitry.
Transceive	1100	Transmits data from FIFO buffer to the antenna and activates automatically the receiver after transmission.
-	1101	Reserved for further use
MFAuthent	1110	Performs the MIFARE® standard authentication as a reader
Soft Reset	1111	Resets the MFRC522

© NXP B.V. 2007. All rights reserved.

pcd_idle (0x00): hủy bỏ lệnh hiện tại, đưa chip về trạng thái nghỉ

pcd_authent (0x0e): thực hiện xác thực mật khẩu (key) để được phép đọc/ghi dữ liệu trên thẻ

pcd_receive (0x08): kích hoạt bộ thu dữ liệu

pcd_transmit (0x04): gửi dữ liệu từ bộ nhớ fifo ra ngoài

pcd_transceive (0x0c): gửi và nhận dữ liệu cùng lúc; đây là lệnh được dùng nhiều nhất để giao tiếp với thẻ

pcd_resetphase (0x0f): reset phần mềm cho chip rc522

pcd_calccrc (0x03): kích hoạt bộ tính toán mã kiểm tra lỗi crc

- Còn các cấu hình còn lại thì em tra trong datasheet và ghi vào

Giải thích file rc522.c

```
#include "rc522.h"
#include "spi.h"
```

- Dòng này có tác dụng thêm các file cấu hình của rc522.h và spi.h

```
extern SPI_HandleTypeDef hspi1;
```

- Dòng này dùng để khai báo biến ngoại lai đã có sẵn trong file spi.c và đem về dùng

```
void MFRC522_WriteRegister(uint8_t addr, uint8_t val) {
    uint8_t addr_bits = (addr << 1) & 0x7E;

    HAL_GPIO_WritePin(RC522_CS_GPIO_Port, RC522_CS_Pin, GPIO_PIN_RESET); //
CS Low
    HAL_SPI_Transmit(&hspi1, &addr_bits, 1, 100);
    HAL_SPI_Transmit(&hspi1, &val, 1, 100);
    HAL_GPIO_WritePin(RC522_CS_GPIO_Port, RC522_CS_Pin, GPIO_PIN_SET);   //
CS High
}
```

- Hàm trên có tác dụng ghi một giá trị cụ thể vào thanh ghi trong chip RC522:

- Đầu tiên là dòng uint8_t addr_bits = (addr << 1) & 0x7E; thì dòng này nó có tác dụng dịch addr qua phải 1 bit, thì dựa vào datasheet:

Table 8. Address byte 0 register; address MOSI

7 (MSB)	6	5	4	3	2	1	0 (LSB)
1 = read 0 = write	address						0

thì thấy từ bit 6 đến 1 là giá trị data nên phép dịch trên có tác dụng đưa giá trị vào đúng ô nhớ. còn &0x7E tức là 01111110 tức là khi và với nó sẽ đưa bit 7 và 0 về giá trị 0 và bit 7 bằng 0 thì tức là bắt đầu ghi

- Sau đó dòng tiếp theo sẽ kéo chân cs xuống mức thấp để bắt đầu giao tiếp với rfid

- Dòng tiếp theo có tác dụng gửi 1 byte địa chỉ addr qua đường MOSI

- Dòng tiếp theo có tác dụng gửi 1 byte dữ liệu nối tiếp byte địa chỉ lúc nãy

- Sau đó dòng cuối có tác dụng kéo chân cs lên mức cao để bỏ chọn

```
uint8_t MFRC522_ReadRegister(uint8_t addr) {
    uint8_t val;
    uint8_t addr_bits = ((addr << 1) & 0x7E) | 0x80;

    HAL_GPIO_WritePin(RC522_CS_GPIO_Port, RC522_CS_Pin, GPIO_PIN_RESET); //
CS Low
    HAL_SPI_Transmit(&hspi1, &addr_bits, 1, 100);
    HAL_SPI_Receive(&hspi1, &val, 1, 100);
    HAL_GPIO_WritePin(RC522_CS_GPIO_Port, RC522_CS_Pin, GPIO_PIN_SET);   //
CS High
    return val;
}
```

- Hàm trên có tác dụng đọc giá trị từ thanh ghi trong chip RC522:

- Dòng này cũng giống hệt hàm bên write nhưng thêm | 0x80 có tức là 10000000 có tác dụng bật bit 7 lên mức 1, theo datasheet đây là lệnh đọc

- Dòng tiếp theo có tác dụng kéo chân CS xuống mức thấp để bắt đầu giao tiếp với rfid qua SPI

- Dòng tiếp theo có tác dụng gửi 1 byte địa chỉ addr qua đường MOSI

- Dòng tiếp theo có tác dụng nhận dữ liệu về thông qua địa chỉ val

- Dòng tiếp theo có tác dụng kéo chân CS lên mức cao để bỏ chọn và dòng cuối sẽ trả về giá trị val đã đọc từ thanh ghi của rfid

```
void MFRC522_SetBitMask(uint8_t reg, uint8_t mask) {
    uint8_t tmp;
    tmp = MFRC522_ReadRegister(reg);
    MFRC522_WriteRegister(reg, tmp | mask);
}

void MFRC522_ClearBitMask(uint8_t reg, uint8_t mask) {
    uint8_t tmp;
    tmp = MFRC522_ReadRegister(reg);
    MFRC522_WriteRegister(reg, tmp & (~mask));
}
```

- Hàm đầu tiên có tác dụng bật các bit chỉ định trong mask lên mức 1 mà vẫn giữ nguyên các bit còn lại:

- Đầu tiên khai báo biến tmp là biến tạm để lưu giá trị đọc được vào

- Tiếp theo là hàm dùng để ghi giá trị, và ở tmp | mask tức là ta thực hiện phép OR giữa giá trị đọc cũ và mặt nạ :

Nguyên lý phép OR:

— x | 1 = 1 (bật bit lên)
— x | 0 = x (giữ nguyên giá trị cũ)

Sau đó ghi kết quả mới quay lại thanh ghi reg

- Còn hàm tiếp theo tương tự nhưng có tác dụng tắt bit chỉ định trong mask về mức 0 mà vẫn giữ nguyên các bit còn lại

- 2 dòng đầu thì tương tự còn dòng cuối có tác dụng ghi , và ở tmp | mask ta thực hiện phép đảo và and có tác dụng:

— đầu tiên ta đảo ngược bit của mask. nơi nào muốn tắt (đang là 1) sẽ biến thành 0, nơi không đụng đến (đang là 0) biến thành 1
— thực hiện phép and
— x & 0 = 0 (bị ép về 0 thì tắt bit)
— x & 1 = x (giữ nguyên giá trị cũ)
— ghi kết quả quay lại thanh ghi

```c
void MFRC522_AntennaOn(void) {
    uint8_t temp;
    temp = MFRC522_ReadRegister(MFRC522_REG_TX_CONTROL);
    if (!(temp & 0x03)) {
        MFRC522_SetBitMask(MFRC522_REG_TX_CONTROL, 0x03);
    }
}

void MFRC522_AntennaOff(void) {
    MFRC522_ClearBitMask(MFRC522_REG_TX_CONTROL, 0x03);
}
```

Hai hàm này có tác dụng để bật và tắt ăng ten thông qua việc gửi mã lệnh đến rfid

Hàm đầu tiên có tác dụng bật ăng ten

- Dòng 1:thì đầu tiên ta sẽ khởi tạo biến temp và gán giá trị đọc được từ thanh ghi điều khiển phát TxControl (có địa chỉ 0x14) nó có tác dụng quản lí phát sóng trên hai chân TX1 và TX2 của chip

- Dòng 2: `if (!(temp & 0x03)) {`

— 0x03 (nhị phân 0000 0011) thì nó là mặt nạ để kiểm tra 2 bit cuối cùng (Bit 1 và Bit 0). Theo datasheet, hai bit này là Tx2RFEn và Tx1RFEn (2 bit bật sóng rf)

Table 47: TxControlReg register (address 14h); reset value: 80h

Bit	7	6	5	4	3	2	1	0
Symbol	InvTx2RF On	InvTx1RF On	InvTx2RF Off	InvTx1RF Off	Tx2CW	-	Tx2RFEn	Tx1RFEn
Access Rights	r/w	r/w	r/w	r/w	r/w	RFU	r/w	r/w

1	Tx2RFEn	Set to logic 1, the output signal on pin TX2 will deliver the 13.56 MHz energy carrier modulated by the transmission data.
0	Tx1RFEn	Set to logic 1, the output signal on pin TX1 will deliver the 13.56 MHz energy carrier modulated by the transmission data.

— temp & 0x03: Lọc lấy trạng thái của 2 bit này

— !(...): Kiểm tra xem kết quả có bằng 0 không

— ý nghĩa: "nếu cả 2 bit này đang tắt (bằng 0) thì ăn ten đang tắt

Dòng 3: `MFRC522_SetBitMask(MFRC522_REG_TX_CONTROL, 0x03);`

— nếu điều kiện trên đúng (ăng-ten đang tắt), hàm này sẽ bật bit 0 và bit 1 lên mức 1

— khi 2 bit này lên 1, chip rc522 bắt đầu phát sóng vô tuyến 13.56mhz ra không gian xung quanh để cấp nguồn cho thẻ

- Còn hàm tắt ăn ten thì đơn giản hơn, nó có tác dụng kéo 2 bit đầu tiên của thanh ghi TxControl xuống mức thấp để tắt

```
void MFRC522_Reset(void) {
    MFRC522_WriteRegister(MFRC522_REG_COMMAND, PCD_RESETPHASE);
}
```

- Đây là hàm dùng để reset, nó có tác dụng ghi giá trị 0x0F (PCD_RESETPHASE) vào thanh ghi lệnh (MFRC522_REG_COMMAND) như hình dưới:

Soft Reset	1111	Resets the MFRC522

```
void MFRC522_Init(void) {
    // Hard Reset qua chân RST
    HAL_GPIO_WritePin(RC522_RST_GPIO_Port, RC522_RST_Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(RC522_RST_GPIO_Port, RC522_RST_Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    MFRC522_Reset();
```

- Đây là hàm dùng để hard reset thông qua việc kéo chân reset xuống mức thức để reset vào sau đó delay 10ms rồi cho lại bình thường

- Còn dòng cuối là gọi lại hàm soft reset để đảm bảo các thanh ghi được sạch trước khi nạp cấu hình mới

```
    // Cấu hình Timer
    MFRC522_WriteRegister(MFRC522_REG_T_MODE, 0x8D);
    MFRC522_WriteRegister(MFRC522_REG_T_PRESCALER, 0x3E);
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_L, 30);
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_H, 0);

    MFRC522_WriteRegister(MFRC522_REG_TX_AUTO, 0x40);
    MFRC522_WriteRegister(MFRC522_REG_MODE, 0x3D);

    MFRC522_AntennaOn();
```

```
}
```

- Đây là đoạn code dùng để cấu hình timer đếm ngược, khi stm32 gửi lệnh hỏi thẻ thì nếu như chờ lâu quá mà không thấy phản hồi và timer đếm về 0 thì stm32 sẽ tự hiểu là không có thẻ

```
    MFRC522_WriteRegister(MFRC522_REG_T_MODE, 0x8D);
    MFRC522_WriteRegister(MFRC522_REG_T_PRESCALER, 0x3E);
```

- dòng đầu thực hiện ghi giá trị 0x8D vào thanh ghi MFRC522_REG_T_MODE hay 2A

Table 91: TModeReg register (address 2Ah); reset value: 00h

Bit	7	6	5	4	3	2	1	0
Symbol	TAuto	TGated		TAutoRestart	TPrescaler_Hi			
Access Rights	r/w	r/w		r/w	r/w			

Table 92: Description of TModeReg bits

Bit	Symbol	Description
7	TAuto	Set to logic 1, the timer starts automatically at the end of the transmission in all communication modes at all speeds. The timer stops immediately after receiving the first data bit if the bit *RxMultiple* in the register *RxModeReg* is not set. If *RxMultiple* is set to logic 1, the timer never stops. In this case the timer can be stopped by setting the bit *TStopNow* in register *ControlReg* to logic 1. Set to logic 0 indicates, that the timer is not influenced by the protocol.
6 to 5	TGated	The internal timer is running in gated mode. **Remark:** In the gated mode, the bit *TRunning* is logic 1 when the timer is enabled by the register bits. This bit does not influence the gating signal. **Value** — **Description** 00 — Non gated mode 01 — Gated by MFIN 10 — Gated by AUX1 11 — Gated by A3
4	TAutoRestart	Set to logic 1, the timer automatically restart its count-down from *TReloadValue*, instead of counting down to zero. Set to logic 0 the timer decrements to 0 and the bit *TimerIRq* is set to logic 1.
3 to 0	TPrescaler_Hi	Defines higher 4 bits for *TPrescaler*. The following formula is used to calculate f_{Timer}: f_{Timer} = 6.78 MHz/TPreScaler. For detailed description see Section 13 "Timer Unit".

- thì dựa vào datasheet ta sẽ chọn 0x8D tức là 0b10001111 thì bit 7 lên 1 tức là timer sẽ bắt đầu đếm khi truyền đi, còn bit 6,5 set 00 tức là không chọn gate mode, còn bit 4 sẽ chọn 0 để timer tự chạy đến 0 rồi dừng, còn 4 bit từ 3 đến 0 là 4 bit cao của bộ chia tần số

- Còn dòng thứ 2 của code là ghi 0x3E vào thanh MFRC522_REG_T_PRESCALER là 8 bit thấp của bộ chia tần số, tức là 2 dòng trên dùng để thiết lập xung nhịp cho timer

```
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_L, 30);
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_H, 0);
```

- 2 dòng này có tác dụng đặt giá trị đếm ngược là 30, tức là nếu vượt quá thì tự timeout

```
    MFRC522_WriteRegister(MFRC522_REG_TX_AUTO, 0x40);
    MFRC522_WriteRegister(MFRC522_REG_MODE, 0x3D);
```

- Còn 2 dòng này dùng để cấu hình sóng và giao thức

- Dòng đầu thì ghi giá trị 0x40 vào thanh ghi MFRC522_REG_TX_AUTO tức là 0x11 thì dựa vào datasheet:

9.2.2.6 TxASKReg

Controls the settings of the transmit modulation.

Table 49: TxAutoReg register (address 15h); reset value: 00h

Bit	7	6	5	4	3	2	1	0
Symbol	-	Force100ASK	-					
Access Rights	RFU	r/w	RFU					

Table 50: Description of TxAutoReg bits

Bit	Symbol	Description
7	-	Reserved for future use.
6	Force100ASK	Set to logic 1, *Force100ASK* forces a 100% ASK modulation independent of the setting in register *ModGsPReg*.
5 to 0	-	Reserved for future use.

- Ta set giá trị 0x40 tức là 01000000 tức là bit 6 lên mức 1 để chọn điều chế ASK

- Dòng sau là ghi giá trị 0x3D hay 00111101 vào thanh ghi MFRC522_REG_MODE tức là:

Table 41: ModeReg register (address 11h); reset value: 3Fh

Bit	7	6	5	4	3	2	1	0
Symbol	MSBFirst	-	TxWaitRF	-	PolMFin	-	CRCPreset	
Access Rights	r/w	RFU	r/w	RFU	r/w	RFU	r/w	

- nó có chức năng là cài đặt CRC là 0x6363. Đây là quy định của chuẩn ISO 14443 để kiểm tra tính toàn vẹn dữ liệu

- Và dòng code cuối có chức năng bật anten dựa vào hàm đã viết hồi nãy

```
uint8_t MFRC522_ToCard(uint8_t command, uint8_t *sendData, uint8_t sendLen,
uint8_t *backData, uint16_t *backLen) {
    uint8_t status = MI_ERR;
    uint8_t irqEn = 0x00;
    uint8_t waitIRq = 0x00;
    uint8_t lastBits;
    uint8_t n;
    uint16_t i;

    switch (command) {
        case PCD_AUTHENT:
            irqEn = 0x12;
            waitIRq = 0x10;
            break;
        case PCD_TRANSCEIVE:
            irqEn = 0x77;
            waitIRq = 0x30;
            break;
        default:
            break;
    }
```

- Hàm trên có tác dụng gửi lệnh và dữ liệu từ stm32 tới module rc522 rồi chờ thẻ phản hồi

- Các tham số đầu vào thì:

— command: lệnh thực hiện
— sendData, sendLen: Dữ liệu muốn gửi đi và độ dài của nó
— backData, backLen: Nơi chứa dữ liệu thẻ trả về và độ dài của nó

```
uint8_t irqEn = 0x00;
    uint8_t waitIRq = 0x00;
```

- Hai dòng code trên có tác dụng khai báo điều khiển ngắt, biến irqEn là biến những ngắt nào cho phép kích hoạt, còn waitIrq thì nó là biến chờ ngắt xảy ra nếu bằng 1 thì mới dừng còn 0 thì chạy tiếp

- Ta có biến irqEn là biến của thanh ghi 0x02

Control bits to enable and disable the passing of interrupt requests.

Table 25. ComIEnReg register (address 02h); reset value: 80h bit allocation

Bit	7	6	5	4	3	2	1	0
Symbol	IRqInv	TxIEn	RxIEn	IdleIEn	HiAlertIEn	LoAlertIEn	ErrIEn	TimerIEn
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

- còn biến waitIrq là biến của thanh ghi 0x04

Interrupt request bits.

Table 29. ComIrqReg register (address 04h); reset value: 14h bit allocation

Bit	7	6	5	4	3	2	1	0
Symbol	Set1	TxIRq	RxIRq	IdleIRq	HiAlertIRq	LoAlertIRq	ErrIRq	TimerIRq
Access	W	D	D	D	D	D	D	D

```
    switch (command) {
        case PCD_AUTHENT:
            irqEn = 0x12;
            waitIRq = 0x10;
            break;
```

- Code trên dùng để kiểm tra key A/B có khớp với thẻ hay không,biến PCD_AUTHENT là 0x0E thì trong datasheet nó là lệnh xác thực:

MFAuthent	1110	performs the MIFARE standard authentication as a reader

còn biến irqEn là 0b00010010 thì tức là bật bit 4 và 1 thì theo datasheet ta có:

Table 25. ComIEnReg register (address 02h); reset value: 80h bit allocation

Bit	7	6	5	4	3	2	1	0
Symbol	IRqInv	TxIEn	RxIEn	IdleIEn	HiAlertIEn	LoAlertIEn	ErrIEn	TimerIEn
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

4	IRq	-	indicates if any interrupt source requests attention with respect to the setting of the interrupt enable bits: see the ComIEnReg and DivIEnReg registers
3	TRunning	1	MFRC522's timer unit is running, i.e. the timer will decrement the TCounterValReg register with the next timer clock **Remark:** in gated mode, the TRunning bit is set to logic 1 when the timer is enabled by TModeReg register's TGated[1:0] bits; this bit is not influenced by the gated signal
2	reserved	-	reserved for future use
1	HiAlert	1	the number of bytes stored in the FIFO buffer corresponds to equation: $HiAlert = (64 - FIFOLength) \leq WaterLevel$ example: FIFO length = 60, WaterLevel = 4 → HiAlert = 1 FIFO length = 59, WaterLevel = 4 → HiAlert = 0
0	LoAlert	1	the number of bytes stored in the FIFO buffer corresponds to equation: $LoAlert = FIFOLength \leq WaterLevel$ example: FIFO length = 4, WaterLevel = 4 → LoAlert = 1 FIFO length = 5, WaterLevel = 4 → LoAlert = 0

- Bit 4 là bật ngắt khi chip rảnh tức là khi xử lí xong lệnh

- Còn bit 1 là bật ngắt khi có lỗi xảy ra

- Còn biến waitIrq là 0x10 tức là 00010000 tức là bật bit 4 lên mức 1 nghĩa là khi nào xử lí xong mã hoá xác thực thì lên mức 1

4	IdleIRq	1	If a command terminates, for example, when the CommandReg changes its value from any command to the Idle command (see Table 149 on page 70) if an unknown command is started, the CommandReg register Command[3:0] value changes to the idle state and the IdleIRq bit is set The microcontroller starting the Idle command does not set the IdleIRq bit

```
        case PCD_TRANSCEIVE:
            irqEn = 0x77;
            waitIRq = 0x30;
            break;
```

- Còn đoạn code trên dùng cho gửi và nhận dữ liệu và mục đích là gửi dữ liệu đi và chờ thẻ trả lời lại ta có biến PCD_TRANSCEIVE là 0x0C thì theo datasheet:

Transceive	1100	transmits data from FIFO buffer to antenna and automatically activates the receiver after transmission

- Ta có biến irqEn lên 0x77 tức là 01110111 thì

— bit 6 (RxIEn): ngắt khi nhận dữ liệu xong
— bit 5 (TxIEn): ngắt khi gửi dữ liệu xong
— bit 4 (IdleIEn): ngắt khi chip rảnh

- Còn 3 bit sau là bit 2, bit 1, bit 0 lần lượt là cảnh báo mức thấp, có lỗi và timer

- Còn biến waitIrq là 0x30 tức là bật bit 5 và 4 thì 2 bit đó theo datasheet nó là:

5	RxIRq	1	receiver has detected the end of a valid data stream if the RxModeReg register's RxNoErr bit is set to logic 1, the RxIRq bit is only set to logic 1 when data bytes are available in the FIFO
4	IdleIRq	1	If a command terminates, for example, when the CommandReg changes its value from any command to the Idle command (see Table 149 on page 70) if an unknown command is started, the CommandReg register Command[3:0] value changes to the idle state and the IdleIRq bit is set The microcontroller starting the Idle command does not set the IdleIRq bit

- Bit 5 là dùng để báo nhận dữ liệu

- Bit 4 thì dùng để báo khi chip rảnh

```
MFRC522_WriteRegister(MFRC522_REG_COMM_IEN, irqEn | 0x80);
    MFRC522_ClearBitMask(MFRC522_REG_COMM_IRQ, 0x80);
    MFRC522_SetBitMask(MFRC522_REG_FIFO_LEVEL, 0x80);

    MFRC522_WriteRegister(MFRC522_REG_COMMAND, PCD_IDLE);
```

- Thì đoạn code trên nó có tác dụng để dọn dẹp bộ nhớ trước khi bắt đầu một lệnh mới

- Thì dòng đầu tiên ghi vào thanh ghi cho phép ngắt(tức là 0x02) giá trị irqEn | 80 thì nó có tác dụng bật thêm bit 7 lên mức 1 thì bit này bật lên để đảm bảo ngắt hoạt động đúng:

Table 26. ComIEnReg register bit descriptions

Bit	Symbol	Value	Description
7	IRqInv	1	signal on pin IRQ is inverted with respect to the Status1Reg register's IRq bit
		0	signal on pin IRQ is equal to the IRq bit; in combination with the DivIEnReg register's IRqPushPull bit, the default value of logic 1 ensures that the output level on pin IRQ is 3-state

```
    MFRC522_ClearBitMask(MFRC522_REG_COMM_IRQ, 0x80);
    MFRC522_SetBitMask(MFRC522_REG_FIFO_LEVEL, 0x80);
```

- Tiếp theo thì 2 hàm này có tác dụng xoá sạch bộ đệm trong thanh ghi

- Hàm đầu tiên ép bit 7 của thanh ghi trên về mức 0 để xoá các bit đang bật:

Table 30. ComIrqReg register bit descriptions

All bits in the ComIrqReg register are cleared by software.

Bit	Symbol	Value	Description
7	Set1	1	indicates that the marked bits in the ComIrqReg register are set
		0	indicates that the marked bits in the ComIrqReg register are cleared

- Còn hàm tiếp theo thì bật giá trị của thanh ghi trên lên mức 1 để clear giá trị đệm FIFO

Table 42. FIFOLevelReg register bit descriptions

Bit	Symbol	Value	Description
7	FlushBuffer	1	immediately clears the internal FIFO buffer's read and write pointer and ErrorReg register's BufferOvfl bit reading this bit always returns 0

```
    MFRC522_WriteRegister(MFRC522_REG_COMMAND, PCD_IDLE);
```

- gửi lệnh trên tức là 0x00 cho thanh ghi lệnh để đưa chip về trạng thái nghĩ

Command	Command code	Action
Idle	0000	no action, cancels current command execution

```
for (i = 0; i < sendLen; i++) {
        MFRC522_WriteRegister(MFRC522_REG_FIFO_DATA, sendData[i]);
    }

    MFRC522_WriteRegister(MFRC522_REG_COMMAND, command);
    if (command == PCD_TRANSCEIVE) {
```

```
        MFRC522_SetBitMask(MFRC522_REG_BIT_FRAMING, 0x80);
    }
```

- Đoạn code trên có chức năng ghi dữ liệu vào FIFO

- Thì vòng lặp trên dùng để ghi từng byte trong mảng sendata vào thanh ghi FIFO, dòng tiếp theo thì có tác dụng ghi mã lệnh vào thanh ghi lệnh (ví dụ PCD_TRANSCEIVE hoặc PCD_AUTHENT)

- Và dòng if cuối thì dùng để xác nhận phát sóng với điều kiện chỉ trao đổi dữ liệu - gửi đi và nhận về thì ta sẽ bật bit 7 của thanh ghi bit framing lên mức 1 để bắt đầu gửi:

Table 48. BitFramingReg register bit descriptions

Bit	Symbol	Value	Description
7	StartSend	1	starts the transmission of data only valid in combination with the Transceive command

```
    i = 2000;
    do {
        n = MFRC522_ReadRegister(MFRC522_REG_COMM_IRQ);
        i--;
    } while ((i != 0) && !(n & 0x01) && !(n & waitIRq));

    MFRC522_ClearBitMask(MFRC522_REG_BIT_FRAMING, 0x80);
```

- đoạn code trên sẽ tạo ra vòng lặp dạng hỏi vòng với biến đếm là 2000, còn biến n sẽ đọc giá trị của thanh ghi điều khiển ngắt, vòng lặp sẽ tiếp tục chạy cho đến khi nào 3 điều kiện trên còn đúng là((i != 0) && !(n & 0x01) && !(n & waitIRq));

- với (i != 0) là chưa hết thời gian chờ

- và !(n & 0x01) là chip RC522 chưa hết giờ nội bộ, tức là khi nào hết giờ thì bit 0 sẽ lên mức 1 và sẽ khiến điều kiện sai để thoát vòng lặp

- với !(n & waitIRq) tức là sẽ chờ sự kiện thẻ trả lời xong thì bit 4 và 5 của n sẽ lên mức 1 và sẽ giống với biến của waitIrq và sau đó sẽ khiến cho điều kiện này sai và thoát vòng lặp

```
    MFRC522_ClearBitMask(MFRC522_REG_BIT_FRAMING, 0x80);
```

- Dòng này có tác dụng tắt bit 7 về mức 0 để tắt chế độ sóng gửi đi

```
if (i != 0) {
        if (!(MFRC522_ReadRegister(MFRC522_REG_ERROR) & 0x1B)) {
            status = MI_OK;
            if (n & irqEn & 0x01) {
                status = MI_NOTAGERR;
            }

            if (command == PCD_TRANSCEIVE) {
                n = MFRC522_ReadRegister(MFRC522_REG_FIFO_LEVEL);
                lastBits = MFRC522_ReadRegister(MFRC522_REG_CONTROL) & 0x07;
                if (lastBits) {
                    *backLen = (n - 1) * 8 + lastBits;
                } else {
                    *backLen = n * 8;
                }

                if (n == 0) {
                    n = 1;
                }
                if (n > 16) { // Giới hạn độ dài đọc về
                    n = 16;
                }

                for (i = 0; i < n; i++) {
                    backData[i] =
MFRC522_ReadRegister(MFRC522_REG_FIFO_DATA);
                }
            }
        } else {
            status = MI_ERR;
        }
    }

    return status;
}
```

- Đoạn code này có tác dụng chờ xem vòng lặp trên kết thúc vì lí do gì đầu tiên ta sẽ kiểm tra nếu (i!=0) tức là có ngắt còn (i = 0) tức là hết thời gian và lỗi nên ta sẽ gán nếu else xảy ra là status là lỗi còn nếu n khác 0 thì ta sẽ:

- Đọc thanh ghi MFRC522_REG_ERROR để lọc cái bit lỗi theo mask là 0x1B tức là 0001 1011:

Table 33. ErrorReg register (address 06h); reset value: 00h bit allocation

Bit	7	6	5	4	3	2	1	0
Symbol	WrErr	TempErr	reserved	BufferOvfl	CollErr	CRCErr	ParityErr	ProtocolErr
Access	R	R	-	R	R	R	R	R

- Thì ta bật các bit trên để kiểm tra các lỗi 0,1,3,4 lần lượt là lỗi giao thức, lỗi kiểm

tra chẵn lẻ, lỗi xung đột(2 thẻ phản hồi 1 lần) và lỗi tràn bộ nhớ đệm và sau đó dùng

! để báo nếu không có lỗi nào thì điều kiện đúng và tiếp tục

- tiếp theo dòng này (n & irqEn & 0x01) dùng để kiểm tra có thẻ hay không, 0x01

tức là bit timerIrq thì nếu bit này bật lên thì tức là bộ đếm giờ bên trong RC522 chạy hết mà thẻ vẫn không phản hồi thì trả về status là không có thẻ

- còn nếu trao đổi dữ liệu không có lỗi thì ta tới với vòng if tiếp theo ta sẽ đọc thanh ghi fifo để kiểm tra xem có bao nhiêu byte dữ liệu mà thẻ vừa gửi sang

- thẻ RFID không phải lúc nào cũng gửi chẵn byte (8 bit). Đôi khi nó gửi 4 bit, 7 bit...

- RxLastBits (trong REG_CONTROL): Cho biết byte cuối cùng có bao nhiêu bit

hợp lệ.

- Công thức:

nếu có bit lẻ (lastBits > 0): Tổng số bit = (Số byte trọn vẹn) * 8 + Số bit lẻ

- nếu không có bit lẻ: Tổng số bit = Số byte * 8

- sau đó kết quả lưu vào backLen để báo cho chương trình chính biết độ dài dữ liệu

```
                if (n == 0) {
                    n = 1;
                }
                if (n > 16) { // Giới hạn độ dài đọc về
                    n = 16;
                }
```

- Còn đoạn code này dùng để giới hạn độ dài đọc về tránh bị tràn bộ đệm, tức là chỉ cho đọc tối đa 16 byte đầu tiên

```
    for (i = 0; i < n; i++) {
    backData[i] = FRC522_ReadRegister(MFRC522_REG_FIFO_DATA);
```

- Sau đó là chạy vòng lặp này để đọc giá trị từ cái thanh ghi fifo để lấy giá trị nó và ghi vào mảng backData đó chính là CardID của thẻ để ta đọc

```
uint8_t MFRC522_Request(uint8_t reqMode, uint8_t *TagType) {
    uint8_t status;
    uint16_t backBits;

    MFRC522_WriteRegister(MFRC522_REG_BIT_FRAMING, 0x07);
    TagType[0] = reqMode;

    status = MFRC522_ToCard(PCD_TRANSCEIVE, TagType, 1, TagType, &backBits);

    if ((status != MI_OK) || (backBits != 0x10)) {
        status = MI_ERR;
    }

    return status;
}
```

- Hàm này có tác dùng dò xem có thẻ nào nằm trong vùng phủ sóng ăn ten hay không

- Sau đó thì nó sẽ ghi giá trị 0x07 vào thanh ghi bit framing tức là ghi vào last bit

Table 47. BitFramingReg register (address 0Dh); reset value: 00h bit allocation

Bit	7	6	5	4	3	2	1	0
Symbol	StartSend	RxAlign[2:0]			reserved	TxLastBits[2:0]		
Access	W	R/W			-	R/W		

thì vì không phải lúc nào cũng truyền đủ 8 bit nên bật 3 bit cuối để chỉ gửi 7 bit do lệnh tìm thẻ nó chỉ dài 7 bit

- và tiếp theo là gán biến regMode là 0x26 tức là lệnh hỏi vào tagtype thì dùng để chuẩn bị gửi đi

- và sau đó ta sẽ gọi hàm MFRC522_ToCard để thực hiện giao tiếp, thì :

+ PCD_TRANSCEIVE: Gửi và nhận

+ 1: Độ dài dữ liệu gửi là 1 byte (nhưng thực tế chỉ phát 7 bit do cài đặt ở bước 1)

+ &backBits: Biến này sẽ hứng độ dài dữ liệu mà thẻ trả lời về (tính bằng bit)

- thì sau đó nó sẽ kiểm tra 2 điều kiện:

+ đầu tiên là status MI_OK tức là không bị lỗi timeout hay lỗi nhiễu

+ thứ hai là kiểm tra backbits nếu khác 0x10 tức là 16 thì khi nhận được lệnh hỏi (0x26), thẻ Mifare bắt buộc phải trả lời một mã gọi là ATQA (Answer To

Request A), thì cái mã trên luôn có độ dài là 16 bit nếu khác thì coi như thẻ không hợp lệ

```
uint8_t MFRC522_Anticoll(uint8_t *serNum) {
    uint8_t status;
    uint8_t i;
    uint8_t serNumCheck = 0;
    uint16_t unLen;

    MFRC522_WriteRegister(MFRC522_REG_BIT_FRAMING, 0x00);
    serNum[0] = PICC_ANTICOLL;
    serNum[1] = 0x20;
    status = MFRC522_ToCard(PCD_TRANSCEIVE, serNum, 2, serNum, &unLen);

    if (status == MI_OK) {
        // Kiểm tra Checksum của UID
        for (i = 0; i < 4; i++) {
            serNumCheck ^= serNum[i];
        }
        if (serNumCheck != serNum[4]) {
            status = MI_ERR;
        }
    }

    return status;
}
```

- Tiếp theo thì hàm này có tác dụng lấy mã định danh uid của thẻ đó

-Đầu tiên ta sẽ ghi 0 vào thanh ghi bit framing để reset lại thành 8 bit do hồi nãy ta phải đổi thành 7 bit theo tiêu chuẩn

- Tiếp theo là 0x93 tức là mã lệnh "Anti-collision Level 1" thì có nghĩa là lệnh để lấy UID, còn 0x20 là tham số NVB tức là thống báo đã gửi mã lệnh trên và ép thẻ gửi toàn bộ UID ngay lập tức

- sau đó thì ta gán status thành giá trị thu được từ hàm MFRC522_toCard, nếu thành công thì status sẽ là MiOk và mảng serNum sẽ chứa 5 byte dữ liệu trả về từ thẻ gồm

+ byte 0 - 3: 4 Byte UID thực sự của thẻ

+ byte 4: Byte kiểm tra checksum

```
    if (status == MI_OK) {
        // Kiểm tra Checksum của UID
        for (i = 0; i < 4; i++) {
            serNumCheck ^= serNum[i];
        }
```

```
        if (serNumCheck != serNum[4]) {
            status = MI_ERR;
        }
    }
```

- thì đoạn code này dùng để kiểm tra checksum, theo chuẩn mifare thì byte thứ 5 luôn bằng kết quả tính xor của 4 byte đầu tiên, nên đoạn code trên sẽ dùng vòng lặp để xor dồn 4 byte đầu tiên rồi kiểm tra với byte cuối

```
void MFRC522_Halt(void) {
    uint16_t unLen;
    uint8_t buff[4];
    buff[0] = PICC_HALT;
    buff[1] = 0;
    MFRC522_ToCard(PCD_TRANSCEIVE, buff, 2, buff, &unLen);
}
```

- hàm này có tác dụng gửi lệnh cho thẻ ngưng hoạt động và chuyển sang trạng thái nghĩ

- ta có buff[0] là PICC_HALT là 0x50 là mã lệnh chuẩn để yêu cầu thẻ ngừng hoạt động

- còn buff[1] là byte tham số đi kèm để gửi và theo tiêu chuẩn thì nó là 0x00

```
uint8_t MFRC522_Check(uint8_t *id) {
    uint8_t status;
    status = MFRC522_Request(PICC_REQIDL, id);
    if (status == MI_OK) {
        status = MFRC522_Anticoll(id);
    }
    MFRC522_Halt();
    return status;
}
```

- Còn hàm trên là hàm check tổng dùng mấy hàm vừa rồi để tạo thành quy trình trọn vẹn để kiểm tra

- Đầu tiên ta sẽ gọi hàm MFRC522_Request với tham số PICC_REQIDL để kiểm tra xung quanh có thẻ ở trạng thái rảnh trong vùng phủ sóng không, nếu có tìm thấy thẻ thì 2 byte đầu tiên của mảng id sẽ chứa Loại thẻ (Tag Type - ATQA)

- Nếu có thẻ rồi thì sau đó ta sẽ gọi hàm anticoll để ghi đè id thành 5 byte của thẻ: là 4 byte số và 1 byte checksum

- Sau đó ta sẽ gọi hàm halt để đưa thẻ vào trạng thái dừng dò thẻ rồi sau đó trả về status

Giải thích file main.c

```
status = MFRC522_Check(CardID);

if (status == MI_OK) {
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
  HAL_Delay(500);
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
```

- Thì đây là đoạn code logic chính của chương trình và để trong vòng lặp while thì đoạn code này sẽ liên tục kiểm tra kết quả của status thông qua hàm check kia và nếu trạng thái trả về là OK thì sẽ đảo trạng thái chân PC13 rồi trả lại để nháy led một cái để báo hiệu, nói chung nếu quét thẻ nhận được uid và không lỗi thì đèn led sẽ nháy lên 1 cái